

Synthetic Meter System & Control Plane — Plan

PREPARED BY Energy Hippo implementation team

The engine that makes the synthetic customer **live**. The data design says **what** each meter reads; this system **emits** those readings into EEMSuite through every gateway's **real intake path** — so we exercise the actual gateway components and honor the "no direct DB seed; feed EEM through real ingestion paths" principle for all time-series/interval/handheld data.

Status: **first AcquiSuite file emitter implemented; remaining emulators and control UI planned**. Start with the [generator instructions](#) and [UI acceptance checklist](#). The supplied normal, gap and backfill files have deterministic expected values; installed ingestion acceptance remains pending.

Grounds: the 15 gateway profiles (21 format cases) plus the HMR handheld-event publisher in `data/scenarios/demo-university-v1.yaml`, EEM's gateway components at `EEMSuite/src/Tasks/EEMSuite.Tasks.Gateways`, and its gateway test fixtures (e.g. `real.MDE` samples under `EEMSuite/tests/.../Gateways/TestData`).

What it is

A **synthetic device/data emulator fleet** + a **control plane**:

- **Meters** = deterministic signal generators (a reading shape per channel).
- **Intake emulators** = adapters that deliver those readings the way each gateway expects (drop a file, answer a device poll, fill a historian table, serve an API).
- **Control plane** = one web UI to turn meters on/off, pick scenarios, set the clock, and run/schedule emission — organized by provider (your "five control systems," unified).

It **never writes to EEM's database**. It only produces inputs EEM's own gateways ingest. That's the whole point — it's how interval/meter data legitimately enters.

Three layers

Layer 1 — Meter model & generation (deterministic)

A meter/channel = { `id`, `provider`, `commodity`, `unit`, `interval`, `shape`, `seed` }. The **shape** composes: baseload + daily curve + seasonal term + weather correlation + occupancy/event modifiers + bounded noise — all deterministic from `seed` + `timestamp`. This reuses the "Required Patterns" already in the data design (seasonal weather correlation, lab baseload > classroom, Student Center event spikes, apartment usage scaling with occupancy, solar midday curve, etc.).

Output is one **canonical reading stream** (`channel_id` × `timestamp` × `value`, plus quality) that every emulator below renders into its target format. Same seed

- date range ⇒ identical readings (the repo's determinism rule).

Layer 2 — Intake emulators (grouped by mechanism)

The important design choice: emulators are built **by intake mechanism (4 kinds), not per provider** — even though the UI groups them by provider for the operator.

Mechanism	Gateways it feeds	Emulator does	Complexity
File-drop	AcquiSuite, MV90 MDEF, MV90 MV9, Spinwave, FIG (7 formats plus the fixed-network submeter file), Neptune (Cedar Row B water route and the Cedar Row apartment water submeter route), MVRs, bill files (BIF/CSV)	write correctly-formatted files into the folder the gateway watches	Medium — format fidelity is the work
Live device (poll)	BACnet, Modbus	stand up a fake BACnet device + a Modbus TCP server that answer EEM's polls with scripted present-values/registers	Higher — must run live while "on"
Database (pull)	ODBC historian	populate a separate historian DB that the ODBC gateway queries (not EEM's DB)	Low-Medium
External API (pull)	NOAA, Aeris	use real public NOAA, or a mock weather endpoint returning scripted obs/forecast	Low (or real)
Internal	Fake	none — EEM's FakeGateway generates its own	None

Format fidelity is the main file-drop challenge. We have templates: EEM's MV90 .MDE test fixtures, and every format is fully defined by its gateway parser in `EEMSuite.Tasks.Gateways`. Plan: clone/derive from the test fixtures + parsers and golden-test each emitter's output against them.

Layer 3 — Control plane (the unified "5 control systems")

One small web UI + backend service:

- **Provider panels** — Valley Electric, Sierra Gas, River City, Northlake Thermal, Helios — plus **Weather / Sensors / Historian** and **Submeters** panels for non-provider sources. This is your "each provider controls its meters," unified in one app.
- **Per meter:** on/off toggle, current scenario (normal | a named anomaly), last-emitted timestamp + status, next scheduled run.
- **Global:** the synthetic **clock / date range**, the **seed**, run-now vs. continuous emission, start/stop the live device emulators.
- **Scenario switches:** the deliberate events become toggles the emitters realize — meter replacement, missing-interval-block + later backfill, estimated → corrected bill, duplicate file import, rate-change boundary, bad-units negative test.

Delivery model: both, batch-first (decided)

Both delivery models are in scope; **batch is built first**.

- **Batch** (generate files/rows for a date range, drop them, let EEM ingest on its schedule) covers **~13 of 15 gateway profiles** — files, historian DB, weather, handheld, bills — and is the path for the **2024–2025 historical backfill**.
- **Live** is required for **BACnet + Modbus**, where EEM **polls a device**: the emulator must be a running process answering reads while the meter is "on."

Batch-first gets the bulk working and gives every point (including the chilled- water/chiller points) its history; the two live device emulators follow as a second phase. The control UI drives both — "run a batch" for batch meters, "start/stop device" for the live two.

Live device emulation (BACnet + Modbus)

The two real-time gateways poll a device, so each needs a small **running server** that answers EEM's reads with Layer-1 readings (same deterministic stream as batch).

- **BACnet emulator** — a BACnet/IP **device** exposing the chilled-water points at Science Center and the Recreation and Aquatics Center (ton-hours, tons) and the Central Plant header points (chilled-water supply, steam production, condensate return) as analog objects. EEM's BACnetGateway discovers/reads present values by the object bindings already in the manifest (ObjectID / Object Type), pointed at the emulator's configured device address (BBMD / device instance). Present values update on the synthetic clock.
- **Modbus emulator** — a Modbus **TCP server** on a configured host:port exposing holding registers with the chiller kW as float32 (correct word order + scaling). EEM's ModbusGateway polls the register addresses in the manifest.

Design notes:

- **Reachability**: both must run where the EEM gateway worker can reach them (same host/network). The control plane manages their lifecycle (start/stop with the meter's on/off).
- **Shared generation**: a BACnet object's value (or Modbus register) at time **T** equals the Layer-1 generated reading for that channel at **T** — determinism holds across batch and live.
- **Point-in-time vs history (important)**: a device poll captures the value **at poll time**; EEM stamps it then. So the live emulators are inherently **forward-going** — they can't replay two years of history. Therefore the **2024–2025 history** for the chilled-water/chiller points is delivered via the **batch path** (interval file or historian), while the live emulators **prove the real-time BACnet/Modbus gateway code paths** and provide ongoing data (a "turn it on and watch EEM poll" demo). **Build-time check**: if EEM reads BACnet **trend logs** / Modbus historical blocks, the emulators could also serve history — confirm against BACnetGateway/ModbusGateway before relying on it.
- **Off-the-shelf**: both protocols have mature server libraries; the emulator wraps one and feeds it Layer-1 values on the clock.

Per-provider control mapping

Provider panel	Meters → intake emulator
Valley Electric	Student Center and Lakeview Residence Hall electric → AcquiSuite file · Science Center, Recreation and Aquatics Center and Central Plant electric → MV90 MDEF file · monthly bills → BIF/CSV
Sierra Gas	Student Center Kitchen Gas → MVRS handheld file · monthly bills → BIF/CSV
River City	Cedar Row A Water Master → FIG files (7 formats) · Cedar Row B Water Master → Neptune handheld · bundled bills → BIF/CSV
Northlake Thermal	Science Center and Recreation and Aquatics Center chilled water, Central Plant production meters → BACnet (live) · chiller kW → Modbus (live) · monthly allocations → file
Helios Solar	Solar generation/export → MV90 MV9 file · PPA invoices → file
Weather / Sensors / Historian	KSAC → NOAA · KSMF → Aeris · lab sensors → Spinwave file · Library electric → ODBC historian · smoke → Fake
Submeters	Cedar Row apartment and house electric submeters, Lakeview Residence Hall laundry, Parking Structure EV chargers → FIG fixed-network submeter file · Cedar Row apartment and house water submeters → Neptune apartment water route file

Determinism, safety, traceability

- Same seed ⇒ same output. Artifacts carry run metadata (commit, scenario, seed) per the repo's artifact-traceability rule.
- **Never** touches EEM's database — only drop folders, the separate historian DB, live device responses, or API mocks. Consistent with the entry-path principle.
- Generated artifacts stay out of git except intentional golden fixtures.

Where it lives / stack (proposal)

- **Generation core + file/DB/API emulators:** extend generators/. The existing generators are Python: `python generators/northlake_packet.py` (with `--check` to validate first) writes the register, coverage, mapping template and the AcquiSuite samples, and `python generators/update_workbook.py` updates the Data Collection workbook. The emulator language is still open.
- **Live device emulators (BACnet/Modbus):** small standalone services using off-the-shelf libraries.
- **Control plane:** a simple local web app + service ("doesn't have to be super complicated" — toggles, a clock, a run button, status).

Build phases

1. **Generation core** — deterministic readings from the manifest (foundation).
2. **File emulators** — AcquiSuite, MV90 ×2, FIG (including the fixed-network submeter file), Spinwave, Neptune (including the Cedar Row apartment water route), MVRS, bills (covers most gateways; golden-tested vs EEM fixtures/parsers).
3. **Historian DB + weather** — ODBC source DB + NOAA/Aeris (pull paths).

4. **Live device emulators** — BACnet + Modbus servers.
5. **Control plane** — the web UI (on/off, scenarios, clock, status), provider panels.
6. **Scenario realization + assertions** — wire the deliberate anomalies to toggles; verify each gateway ingests and the expected totals land.

Open decisions (for when we build)

- ✓ **Delivery model** — DECIDED: build **both, batch-first** (live BACnet/ Modbus emulators follow).
1. **Stack & home** — extend generators/ in this repo; pick a language; control-plane UI framework.
 2. **Weather** — real public NOAA vs. a mock endpoint (Aeris needs a mock or creds).
 3. **Control-plane fidelity** — simple on/off + scenario toggles vs. richer scripting.
 4. **Live device hosting** — where the BACnet/Modbus emulators run so EEM can reach them (same host/network as the gateway worker).